



AGRUPAMENTO DE ESCOLAS
EDUARDO GAGEIRO

Escola Secundária de Sacavém

**CURSO PROFISSIONAL DE GESTÃO E
PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS**

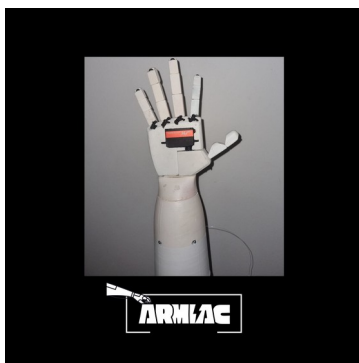
PROVA DE APTIDÃO PROFISSIONAL

Relatório de Projeto

Ano Letivo 2025/2026

Relatório de Projeto

ARMIAC – Braço Inteligente



Quévin Leonardo Aguiar Tavares

2025/2026

Professoras Orientadoras: Cláudia Barata & Paula Alcobia

Índice

1 – Introdução.....	4
2 – Tecnologias Envolvidas.....	5
3 – Implementação do Projeto.....	6
3.1 – Cronograma de desenvolvimento do projeto.....	6
3.2 – Aspetos técnicos do desenvolvimento do trabalho.....	6
4 – Conclusão.....	7
5 – Bibliografia/ Webgrafia.....	8
6 – Anexos.....	9

Índice de Figuras

Figura 1: Tecnologias no nosso quotidiano.....4

Agradecimentos

Em primeiro lugar, gostaria de agradecer às professoras orientadoras **Cláudia Barata** e **Paula Alcobia**, pelo acompanhamento, apoio e sugestões ao longo do desenvolvimento do projeto.

Agradeço também a todos os professores que contribuíram para a minha formação ao longo do Curso Profissional de Gestão e Programação de Sistemas Informáticos, fornecendo conhecimentos importantes para a realização deste trabalho, nomeadamente ao professor **Luís Roque** por fornecer acesso a sala de impressão 3d disponível da escola.

Não posso deixar de agradecer à minha família, especialmente a minha mãe pelo apoio e incentivo durante todo o processo de desenvolvimento do projeto.

Por fim, agradeço a todos os colegas e pessoas que de alguma forma contribuíram para o desenvolvimento do ARMIAC, através de opiniões, ideias e feedback.

1 – Introdução

A PAP (Prova de Aptidão Profissional) é um projeto final de curso que tem como objetivo demonstrar as competências adquiridas ao longo dos três anos de formação no Curso Profissional de Gestão e Programação de Sistemas Informáticos.

O projeto escolhido para a apresentação da PAP intitula-se: “**ARMIAC – Braço Inteligente**”.

A escolha deste projeto surgiu devido ao meu interesse pela robótica e pela criação de sistemas automatizados capazes de simular movimentos humanos. O objetivo foi desenvolver um protótipo funcional de um braço robótico, capaz de executar movimentos de dedos e gestos, sendo controlado remotamente através de uma aplicação móvel, por voz, por controle e por câmara.

O ARMIAC consiste numa mão robótica impressa em 3D, controlada por servomotores, onde os dedos se movimentam através de um sistema de fios semelhante ao funcionamento de tendões humanos. O sistema é controlado por um microcontrolador **ESP32**, utilizando um módulo **PCA9685** para gerir vários servos ao mesmo tempo. Para o controlo remoto foi desenvolvida uma aplicação em **Dart/Flutter**, que comunica com o ESP32 via Wi-Fi, através de pedidos HTTP.

Este projeto foi desenvolvido em diferentes fases:

- 1ª Fase – Pesquisa e planeamento:** Análise de ideias e estudo de projetos semelhantes;
- 2ª Fase – Desenvolvimento mecânico:** Escolha, impressão 3D e montagem das peças;
- 3ª Fase – Implementação eletrónica:** Instalação de servos, cabos e alimentação;
- 4ª Fase – Programação do ESP32:** Controlo PWM e criação de servidor web;
- 5ª Fase – Desenvolvimento da aplicação Flutter:** Interface e comunicação.
- 6ª Fase – Testes finais:** Ajustes de movimentos e melhorias.

Ao longo deste relatório serão apresentadas as tecnologias utilizadas, os materiais aplicados, o desenvolvimento do projeto, a explicação do código e uma conclusão final com melhorias futuras.

Figura 1: Braço robótico ARMIAC



2 – Tecnologias Envolvidas

Para a elaboração deste projeto foi necessário utilizar diversas tecnologias e ferramentas, tanto ao nível de software como de hardware.

2.1 – Software utilizado

Software para escrita do relatório:

- LibreOffice

Software para programação:

- Arduino IDE
- Visual Studio Code / Android Studio

Software para impressão 3D:

- Elegoo Cura/Ultimaker Cura

Software para criação gráfica:

- Canva (logótipo do ARMIAC e mapa representativo)

Figura 2: Arduino IDE utilizado para programação do ESP32



Figura 3: Visual Studio Code utilizado para programação da aplicação Flutter

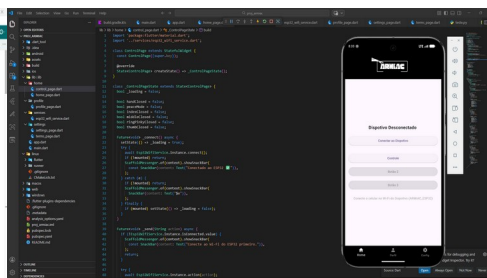
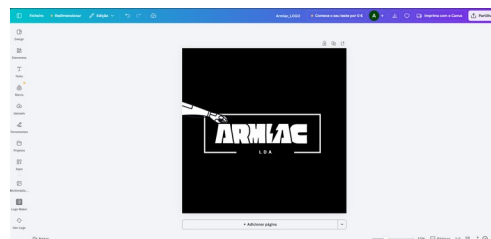


Figura 4: Criação do logótipo ARMIAC no Canva



2.2 – Linguagens de Programação

- **C/C++** (Arduino / ESP32)
- **Dart** (Flutter)

2.3 – Tecnologias e comunicação

- Wi-Fi (ESP32 em modo Access Point)
- Protocolo HTTP (comandos enviados por URL)
- I2C (comunicação ESP32 ↔ PCA9685)
- PWM (controlo dos servos)

2.4 – Componentes eletrónicos utilizados

ESP32

Microcontrolador utilizado como controlador principal do sistema, responsável por criar uma rede Wi-Fi e receber comandos enviados pela aplicação.

Figura 5: Microcontrolador ESP32



PCA9685

Driver PWM de 16 canais utilizado para controlar vários servos ao mesmo tempo com maior estabilidade.

Figura 6: Módulo PCA9685 (Driver PWM 16 canais)



Servomotores MG996R

Servos de torque elevado utilizados para movimentar os dedos e o antebraço.

Figura 7: Servo MG996



Servo JX (baixo perfil)

Servo utilizado especificamente para o movimento do polegar para dentro.

Figura 8: Servo JX (baixo perfil)



Fonte DC 6V 10A

Fonte utilizada para alimentar todos os servomotores e garantir estabilidade de corrente.

Figura 9: Fonte de alimentação DC 6V
10A

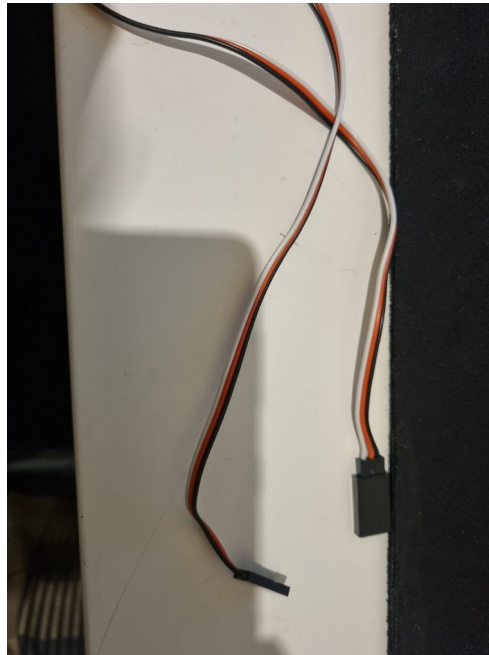


2.5 – Cablagem e acessórios de ligação

Para a montagem e ligação dos servomotores foi necessário utilizar cabos e extensões, garantindo uma ligação correta entre os servos e o PCA9685.

Foram utilizados cabos de extensão de servo principalmente para o servo low profile do polegar, pois o posicionamento deste servo exigiu maior comprimento de cabo.

Figura 10: Cabos de extensão utilizados para ligação do servo do polegar



2.6 – Ferramentas de medição e teste (Multímetro)

Durante o desenvolvimento do projeto foi utilizado um multímetro digital para medir a voltagem e garantir segurança antes de ligar os componentes à fonte de alimentação.

Esta etapa foi importante para evitar danos nos componentes eletrônicos, assegurando que a fonte fornecia o valor correto de tensão antes da ligação final.

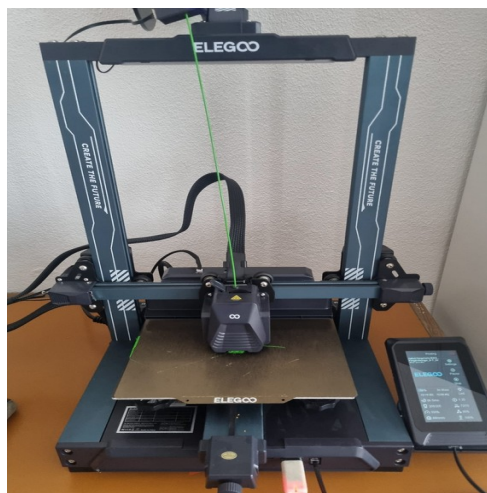
Figura 11: Medição de tensão com multímetro antes de ligar o sistema



2.7 – Impressão 3D e materiais de montagem

O protótipo foi construído com peças impressas em 3D numa impressora **Elegoo Neptune 4**.

Figura 12: Impressora Elegoo Neptune 4



2.8 – Materiais de montagem

Materiais utilizados:

- fio elástico preto 3 mm
- fio de nylon 0,4 mm
- parafusos
- supercola

Figura 13: Materiais de montagem utilizados no projeto



3 – Implementação do Projeto

3.1 – Enquadramento teórico

Braço robótico

Um braço robótico é um sistema mecânico e eletrónico capaz de simular movimentos humanos. Estes sistemas são frequentemente utilizados em áreas industriais, médicas e tecnológicas.

No projeto ARMIAC foi desenvolvido um protótipo de braço robótico com mão articulada, permitindo movimentos de abertura e fecho dos dedos.

Servomotores

Os servomotores permitem controlar um ângulo com precisão, geralmente entre 0° e 180°. O ARMIAC utiliza servos para controlar os movimentos de cada dedo.

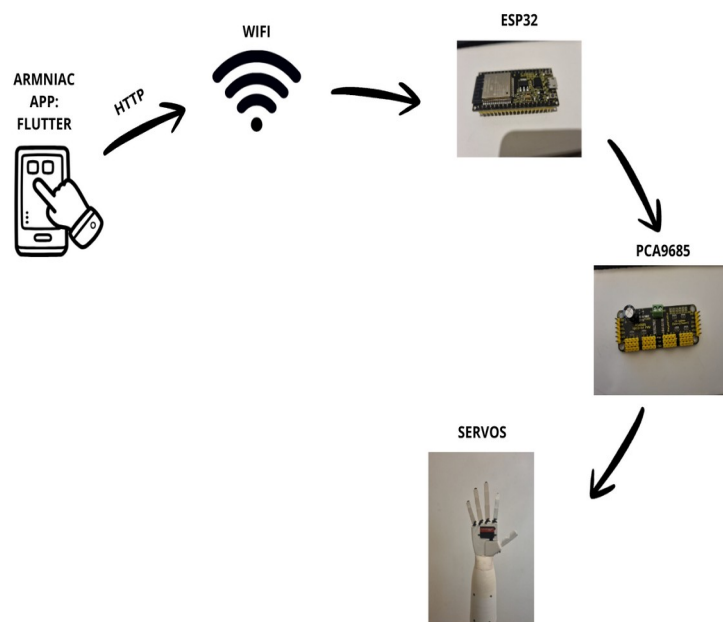
PWM

PWM (Pulse Width Modulation) é uma técnica utilizada para controlar a posição dos servos através de sinais elétricos com larguras de pulso diferentes.

Comunicação por Wi-Fi e HTTP

O controlo remoto é feito através de uma rede Wi-Fi criada pelo ESP32. A aplicação Flutter envia comandos HTTP que são interpretados pelo microcontrolador.

Figura 14: Mapa representativo



4 – Desenvolvimento do Projeto

4.1 – Cronograma e fases de desenvolvimento

O desenvolvimento do projeto **ARMIAC – Braço Inteligente** foi realizado ao longo do ano letivo 2025/2026, seguindo uma sequência de etapas organizadas, desde a fase inicial de pesquisa até ao desenvolvimento da aplicação móvel e testes finais do protótipo.

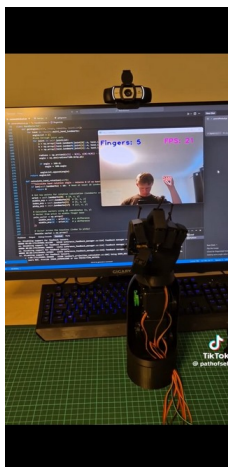
O projeto foi planeado de forma progressiva, permitindo primeiro estruturar a ideia e estudar possíveis soluções técnicas, avançando depois para a impressão 3D, montagem física e eletrónica, e por fim para a implementação do software.

As fases principais do projeto foram as seguintes:

Fase 1 – Pesquisa e planeamento (05 de outubro)

No dia **05 de outubro**, iniciou-se a fase de pesquisa e planeamento, onde foram analisadas ideias e estudados projetos semelhantes, com o objetivo de compreender os componentes necessários e como seria possível desenvolver uma mão robótica funcional.

Figura 15: Primeiro contacto com este tipo de projecto



Fase 2 – Impressão das peças 3D (20 de outubro)

No dia **20 de outubro**, iniciou-se a impressão 3D das peças do projeto. Nesta fase foram produzidas as primeiras estruturas necessárias para a montagem da mão e antebraço, utilizando 2 impressoras **Elegoo Neptune 4**.

Figura 16: Impressão de peças 3D



Fase 3 – Pintura das peças (23 de outubro)

No dia **23 de outubro**, foi realizada a pintura das peças impressas, com o objetivo de melhorar o aspeto visual do protótipo e dar uma aparência mais profissional ao projeto.

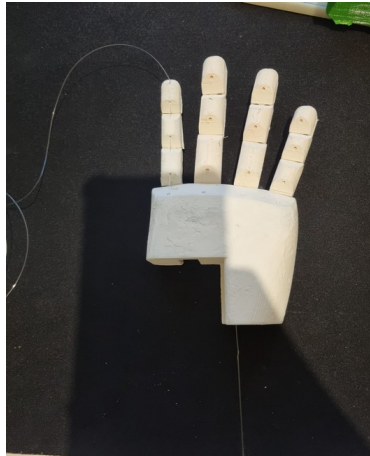
Figura 17: Pintura das peças 3D



Fase 4 – Montagem inicial da mão (29 de outubro)

No dia **29 de outubro**, iniciou-se a montagem da mão robótica, incluindo a montagem das articulações dos dedos e o ajuste inicial das peças impressas.

Figura 18: Início da montagem da mão

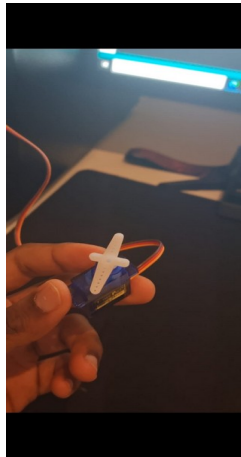


Fase 5 – Primeiro teste com servomotor (05 de novembro)

No dia **05 de novembro**, foi realizado o primeiro teste com um mini servomotor, com o objetivo de validar o funcionamento mecânico do sistema e testar a capacidade de movimentação de um dedo.

Este teste foi essencial para confirmar que o mecanismo de fios e articulações funcionava corretamente.

Figura 19: Primeiro teste com servo



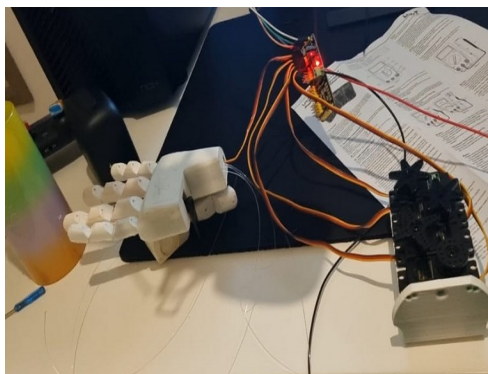
[Link vídeo de teste](#)

Fase 6 – Testes e montagem do antebraço (14 a 16 de novembro)

Entre os dias **14 e 16 de novembro**, foram realizados vários testes adicionais e iniciou-se a montagem do restante antebraço, incluindo a instalação de mais servos e a organização do espaço interno.

Durante esta fase foram feitos ajustes no sistema de fios e nas ligações elétricas, garantindo maior estabilidade e funcionalidade do protótipo.

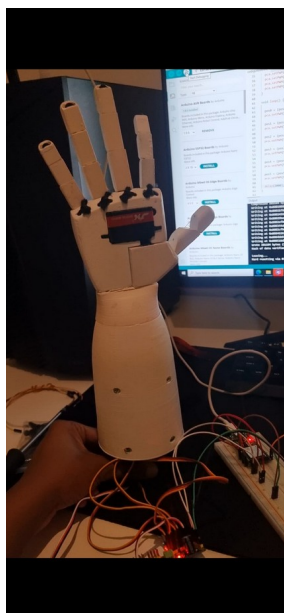
Figura 20: Parte interna do braço



Fase 7 – Montagem final do antebraço (20 de janeiro)

No dia **20 de janeiro**, o antebraço encontrava-se já totalmente montado, incluindo a instalação dos servomotores, fixação das peças e preparação para testes finais e calibração.

Figura 21: Braço montado



Fase 8 – Desenvolvimento da aplicação móvel (21 de janeiro)

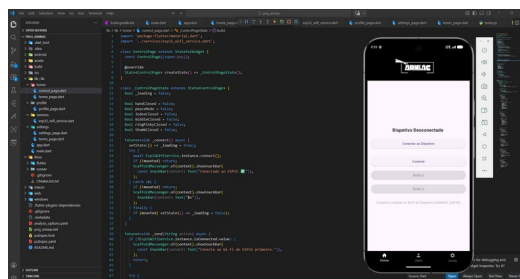
No dia **21 de janeiro**, iniciou-se o desenvolvimento da aplicação móvel em Flutter, com o objetivo de criar uma interface simples e funcional para o controlo remoto do braço robótico através de Wi-Fi.

Nesta fase foram criadas páginas de controlo, botões de ações e um sistema de comunicação HTTP entre o telemóvel e o ESP32.

Figura 22: App no telemóvel



Figura 23: Programação da App flutter



Fase 9 – Demonstração APP (10 de fevereiro)

No dia **10 de fevereiro** foi realizada a demonstração do protótipo ARMIAC juntamente com a aplicação móvel desenvolvida em Flutter. Nesta fase foram efetuados testes finais ao sistema completo, verificando a comunicação Wi-Fi entre o telemóvel e o ESP32 e a resposta dos servomotores aos comandos enviados pela aplicação.

Após a validação do funcionamento do protótipo, foi gravado um vídeo de demonstração onde é possível observar o controlo do braço robótico através da aplicação e a execução dos movimentos e gestos configurados.

[Link para observar](#)

4.2 – Elaboração do protótipo

4.2.1 – Preparação dos modelos 3D

Nesta fase foram criados e ajustados modelos 3D para impressão das peças necessárias para o projeto, incluindo a estrutura da mão e do antebraço em STL. Os ficheiros foram exportados em formato **gcode**

Figura 24: Base da mão

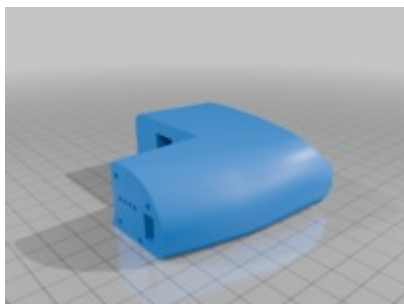


Figura 25: Dedos

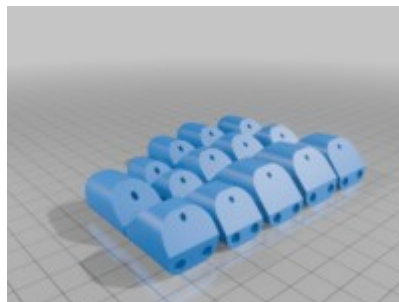


Figura 26: Dedo opositor

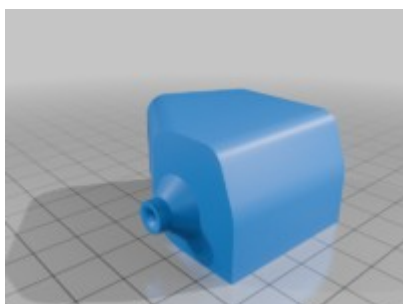


Figura 27: Antebraço (parte 1)

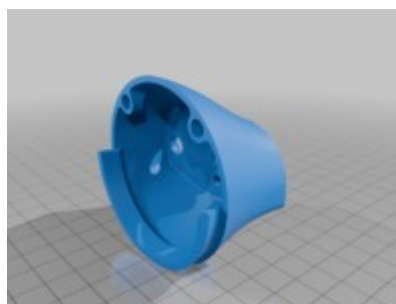


Figura 28: Antebraço (parte 2)

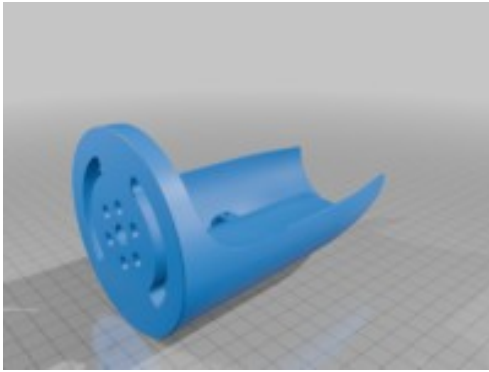


Figura 29: Antebraço (parte 3)

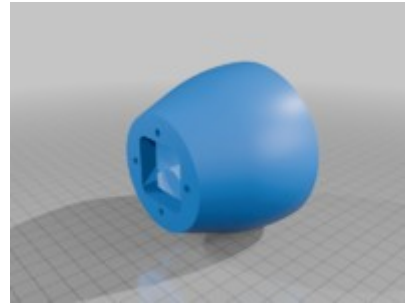


Figura 30: Suporte dos servos

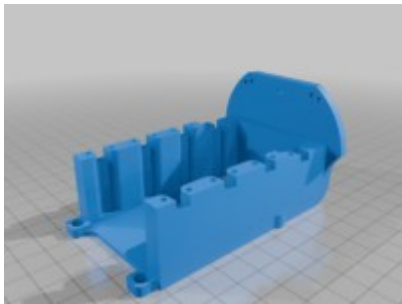
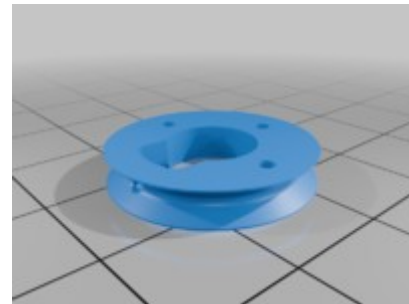


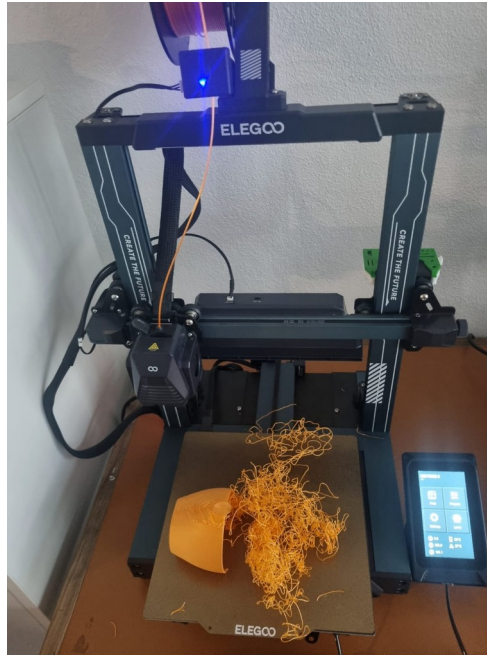
Figura 31: Attach dos servos



4.2.2 – Impressão 3D das peças

Após a criação e preparação dos modelos 3D, foi realizada a impressão das peças utilizando a impressora Elegoo Neptune 4.

Figura 32: Erro na impressão do antebraço



4.2.3 – Montagem dos dedos e articulações

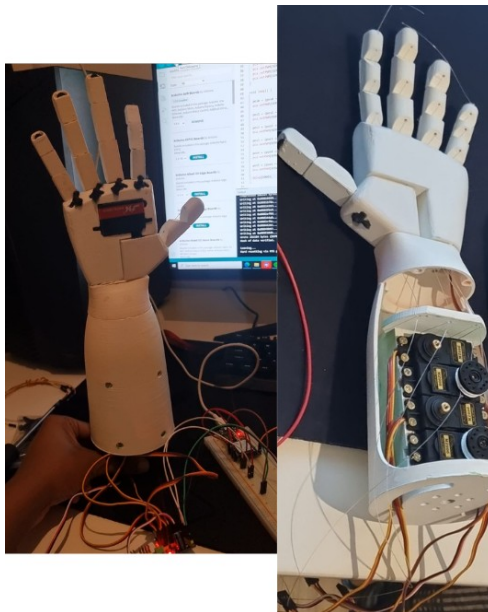
Os dedos foram montados com parafusos e articulações que permitem flexão, simulando o movimento natural de uma mão humana.

[Link para o vídeo a demonstrar](#)

4.2.4 – Instalação dos servomotores

Foram instalados vários servomotores MG996R para controle dos dedos e rotação do antebraço, bem como um servo low profile para o polegar.

Figura 33: Demonstração servos

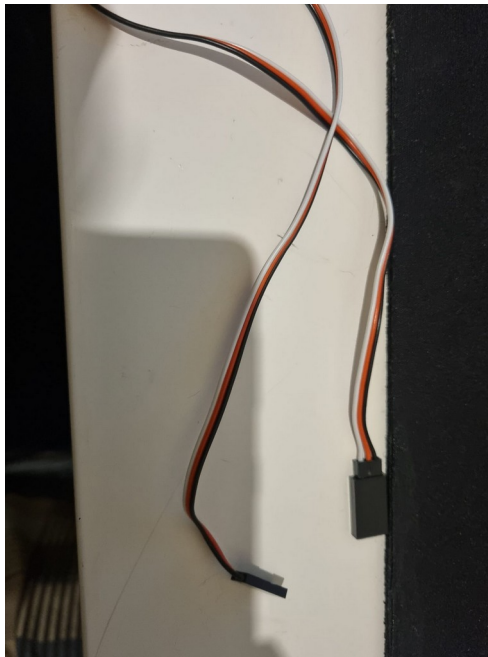


4.2.5 – Cablagem e extensões

Devido à distância entre alguns servos e o controlador PCA9685, foram utilizadas extensões de cabos de servo, especialmente para o servo low profile do polegar.

Esta etapa foi importante para garantir estabilidade nas ligações e organização interna do braço.

Figura 34: Cabos de extensão utilizados para ligação do servo do polegar



4.2.7 – Testes elétricos e medições com multímetro

Antes de ligar o sistema completo à fonte de alimentação, foi utilizado um multímetro para medir a voltagem e garantir que o valor se encontrava correto (6V). Esta verificação permitiu aumentar a segurança do sistema e reduzir riscos de danos nos componentes.

Figura 35: Medição de tensão com multímetro antes de ligar o sistema



4.2.8 – Alimentação e primeiros testes

Após a montagem mecânica e eletrônica, foi realizada a alimentação do sistema utilizando a fonte DC de 6V 10A. Foram feitos testes iniciais para verificar se todos os servos respondiam corretamente aos comandos.

[Link para ver demonstração](#)

4.3 – Programação do ESP32 (Arduino IDE)

A programação do ESP32 foi feita no Arduino IDE. O microcontrolador foi configurado para criar uma rede Wi-Fi (Access Point) e receber comandos através de um servidor web.

Foram criadas rotas HTTP para permitir o controle remoto do braço robótico através da aplicação Flutter.

Código ESP32(C++):

```
#include <Wire.h>
#include <WiFi.h>
#include <WebServer.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pca = Adafruit_PWMServoDriver(0x40);
WebServer server(80);

const char* ssid = "ARMIAC_ESP32";
const char* pass = "12345678";

#define SERVO_MIN 110
#define SERVO_MAX 500

#define CH_POLEGAR_DENTRO 0
#define CH_POLEGAR_FLEX 1
#define CH_INDICADOR 2
#define CH_MEDIO 3
```

```

#define CH_RINGPINKY    4
#define CH_SERVO6      5

int angServo6 = 0;
bool subindo6 = true;
unsigned long ultimoMov6 = 0;
const unsigned long intervalo6_ms = 20;
const int LIM_INF6 = 0;
const int LIM_SUP6 = 180;

uint16_t anguloParaPulso(int angulo) {
    return map(angulo, 0, 180, SERVO_MIN, SERVO_MAX);
}

void setServoAng(int ch, int ang) {
    ang = constrain(ang, 0, 180);
    pca.setPWM(ch, 0, anguloParaPulso(ang));
}

int angInvertido(int fechado) {
    return fechado ? 0 : 180;
}

void setIndicador(int fechado) {
    setServoAng(CH_INDICADOR, fechado ? 180 : 0);
}

void setMedio(int fechado) {
    setServoAng(CH_MEDIO, angInvertido(fechado));
}

void setRingPinky(int fechado) {
    setServoAng(CH_RINGPINKY, angInvertido(fechado));
}

void setPolegarFlex(int fechado) {
    setServoAng(CH_POLEGAR_FLEX, angInvertido(fechado));
}

void setPolegarDentro(int fechado) {
    int ang = fechado ? 0 : 48;
    setServoAng(CH_POLEGAR_DENTRO, ang);
}

void actionCloseHand() {
    setIndicador(1);
    setMedio(1);
}

```

```

    setRingPinky(1);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

void actionPeace() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(1);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

void actionCloseIndex() {
    setIndicador(1);
}

void actionCloseMiddle() {
    setMedio(1);
}

void actionCloseRingPinky() {
    setRingPinky(1);
}

void actionCloseThumb() {
    setPolegarFlex(1);
    setPolegarDentro(1);
}

void actionOpenHand() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(0);
    setPolegarFlex(0);
    setPolegarDentro(0);
}

void actionOpenIndex() {
    setIndicador(0);
}

void actionOpenMiddle() {
    setMedio(0);
}

void actionOpenRingPinky() {

```

```

    setRingPinky(0);
}

void actionOpenThumb() {
    setPolegarFlex(0);
    setPolegarDentro(0);
}

void handlePing() {
    server.send(200, "text/plain", "OK");
}

void handleAction() {
    String a = server.arg("a");

    if (a == "close_hand") actionCloseHand();
    else if (a == "peace") actionPeace();
    else if (a == "close_index") actionCloseIndex();
    else if (a == "close_middle") actionCloseMiddle();
    else if (a == "close_ringpinky") actionCloseRingPinky();
    else if (a == "close_thumb") actionCloseThumb();
    else if (a == "open_hand") actionOpenHand();
    else if (a == "open_index") actionOpenIndex();
    else if (a == "open_middle") actionOpenMiddle();
    else if (a == "open_ringpinky") actionOpenRingPinky();
    else if (a == "open_thumb") actionOpenThumb();
    else {
        server.send(400, "text/plain", "Ação inválida");
        return;
    }

    server.send(200, "text/plain", "OK");
}

void setup() {
    Serial.begin(115200);

    Wire.begin(21, 22);

    pca.begin();
    pca.setPWMPFreq(50);
    delay(10);

    setServoAng(CH_INDICADOR, 180);
    setServoAng(CH_POLEGAR_FLEX, 180);
    setServoAng(CH_MEDIO, 180);
    setServoAng(CH_RINGPINKY, 180);
}

```

```

angServo6 = LIM_INF6;
setServoAng(CH_SERVO6, angServo6);

WiFi.mode(WIFI_AP);
WiFi.softAP(ssid, pass);

Serial.print("AP: ");
Serial.println(ssid);

Serial.print("IP: ");
Serial.println(WiFi.softAPIP());

server.on("/ping", handlePing);
server.on("/action", handleAction);

server.begin();

Serial.println("READY");
}

void loop() {
  unsigned long agora = millis();

  if (agora - ultimoMov6 >= intervalo6_ms) {
    ultimoMov6 = agora;

    setServoAng(CH_SERVO6, angServo6);

    if (subindo6) {
      angServo6++;
      if (angServo6 >= LIM_SUP6) {
        angServo6 = LIM_SUP6;
        subindo6 = false;
      }
    } else {
      angServo6--;
      if (angServo6 <= LIM_INF6) {
        angServo6 = LIM_INF6;
        subindo6 = true;
      }
    }
  }
}

server.handleClient();
}

```


4.4 – Programação da aplicação Flutter

Para o controlo do ARMIAC foi desenvolvida uma aplicação móvel em Flutter, permitindo enviar comandos HTTP ao ESP32.

A aplicação possui uma página inicial com o estado de ligação e uma página de controlo onde é possível abrir/fechar a mão e controlar os dedos individualmente.

Figura 36: App no telemóvel



Figura 37: Controle do ARMIAC na app



Código Flutter(Dart):

Diretório – lib/main.dart

```
import 'package:flutter/material.dart';
import 'app.dart';

void main() {
  runApp(const MyApp());
}
```

Diretório – lib/app.dart

```
import 'package:flutter/material.dart';
import 'home/home_page.dart';
import 'profile/profile_page.dart';
import 'settings/settings_page.dart';

class MyApp extends StatelessWidget {

  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
```

```

return MaterialApp(

  debugShowCheckedModeBanner: false,

  title: 'Armiac',

  theme: ThemeData(

    colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
    useMaterial3: true,

  ),

  home: const MyHomePage(),
);
}
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {

  int _selectedIndex = 0;

  final List<Widget> _pages = const [
    HomePage(),
    ProfilePage(),
    SettingsPage(),
  ];

  void _onItemTapped(int index) => setState(() => _selectedIndex = index);

  @override
  Widget build(BuildContext context) {

    return Scaffold(

      appBar: AppBar(

        backgroundColor: Colors.black,

```

```

centerTitle: true,

toolbarHeight: 120,

title: SizedBox(

  height: 200,

  child: Image.asset(
    "assets/images/Armiac_LOGO.png",

    fit: BoxFit.contain,
  ),
),
),

body: _pages[_selectedIndex],

bottomNavigationBar: BottomNavigationBar(

  currentIndex: _selectedIndex,

  onTap: _onItemTapped,

  backgroundColor: Colors.black,

  selectedItemColor: Colors.white,

  unselectedItemColor: Colors.grey,

  items: const [

    BottomNavigationBarItem(
      icon: Icon(Icons.home),
      label: "Home",
    ),

    BottomNavigationBarItem(
      icon: Icon(Icons.person),
      label: "Perfil",
    ),

    BottomNavigationBarItem(
      icon: Icon(Icons.settings),
      label: "Config",
    ),
  ],

```

```

    ),
  );
}
}

```

Diretório – lib/home/control_page.dart

```

import 'package:flutter/material.dart';
import '../services/esp32_wifi_service.dart';

class ControlPage extends StatefulWidget {
  const ControlPage({super.key});

  @override
  State<ControlPage> createState() => _ControlPageState();
}

class _ControlPageState extends State<ControlPage> {
  bool _loading = false;

  bool handClosed = false;
  bool peaceMode = false;
  bool indexClosed = false;
  bool middleClosed = false;
  bool ringPinkyClosed = false;
  bool thumbClosed = false;

  Future<void> _connect() async {
    setState(() => _loading = true);
    try {
      await Esp32WifiService.instance.connect();
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text("Conectado ao ARMIAC")),
      );
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text("$e")),
      );
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _send(String action) async {
    if (!Esp32WifiService.instance.isConnected.value) {
      ScaffoldMessenger.of(context).showSnackBar(

```

```

        const SnackBar(content: Text("Conecte ao Wi-Fi do ARMIAC primeiro.")),
    );
    return;
}

try {
    await Esp32WifiService.instance.action(action);
} catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text("Erro: $e")),
    );
}
}

Future<void> _resetAll() async {
    await _send("open_hand");

    setState(() {
        handClosed = false;
        peaceMode = false;
        indexClosed = false;
        middleClosed = false;
        ringPinkyClosed = false;
        thumbClosed = false;
    });
}

Widget _toggleButton({
    required String label,
    required bool state,
    required VoidCallback onPressed,
}) {
    return SizedBox(
        width: double.infinity,
        child: ElevatedButton(
            onPressed: onPressed,
            style: ElevatedButton.styleFrom(
                backgroundColor: state ? Colors.green : null,
            ),
            child: Text(state ? "$label " : label),
        ),
    );
}

@override
Widget build(BuildContext context) {
    return Scaffold(

```

```

appBar: AppBar(title: const Text("Controle da Mão")),
body: Padding(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: [
      SizedBox(
        width: double.infinity,
        child: ElevatedButton(
          onPressed: _loading ? null : _connect,
          child: Text(_loading ? "Conectando..." : "Conectar ao ARMIAC"),
        ),
      ),
      const SizedBox(height: 14),

      SizedBox(
        width: double.infinity,
        child: ElevatedButton(
          onPressed: _resetAll,
          style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
          child: const Text("RESETAR (Abrir mão)"),
        ),
      ),
      const SizedBox(height: 20),

      _toggleButton(
        label: "Fechar mão",
        state: handClosed,
        onPressed: () async {
          if (!handClosed) {
            await _send("close_hand");
          } else {
            await _send("open_hand");
          }
        },
      ),

      setState(() {
        handClosed = !handClosed;

        // se fechar mão, desativa outros modos
        if (handClosed) {
          peaceMode = false;
          indexClosed = true;
          middleClosed = true;
          ringPinkyClosed = true;
          thumbClosed = true;
        } else {
          indexClosed = false;

```

```

        middleClosed = false;
        ringPinkyClosed = false;
        thumbClosed = false;
    }
  });
},
),

```

```

const SizedBox(height: 10),

```

```

_toggleButton(
  label: "Símbolo paz",
  state: peaceMode,
  onPressed: () async {
    if (!peaceMode) {
      await _send("peace");
    } else {
      await _send("open_hand");
    }
  }

```

```

  setState(() {
    peaceMode = !peaceMode;

```

```

    if (peaceMode) {
      handClosed = false;
      indexClosed = false;
      middleClosed = false;
      ringPinkyClosed = true;
      thumbClosed = true;
    } else {
      ringPinkyClosed = false;
      thumbClosed = false;
    }
  });
},
),

```

```

const SizedBox(height: 10),

```

```

_toggleButton(
  label: "Dedo indicador",
  state: indexClosed,
  onPressed: () async {
    if (!indexClosed) {
      await _send("close_index");
    } else {
      await _send("open_index");
    }
  }

```



```

    }
    setState(() {
      indexClosed = !indexClosed;
      handClosed = false;
      peaceMode = false;
    });
  },
),

```

```

const SizedBox(height: 10),

```

```

_toggleButton(
  label: "Dedo médio",
  state: middleClosed,
  onPressed: () async {
    if (!middleClosed) {
      await _send("close_middle");
    } else {
      await _send("open_middle");
    }
    setState(() {
      middleClosed = !middleClosed;
      handClosed = false;
      peaceMode = false;
    });
  },
),

```

```

const SizedBox(height: 10),

```

```

_toggleButton(
  label: "Dedo anelar + mindinho",
  state: ringPinkyClosed,
  onPressed: () async {
    if (!ringPinkyClosed) {
      await _send("close_ringpinky");
    } else {
      await _send("open_ringpinky");
    }
    setState(() {
      ringPinkyClosed = !ringPinkyClosed;
      handClosed = false;
      peaceMode = false;
    });
  },
),

```

```

const SizedBox(height: 10),

_toggleButton(
  label: "Dedão",
  state: thumbClosed,
  onPressed: () async {
    if (!thumbClosed) {
      await _send("close_thumb");
    } else {
      await _send("open_thumb");
    }
    setState(() {
      thumbClosed = !thumbClosed;
      handClosed = false;
      peaceMode = false;
    });
  },
),
],
),
),
);
}
}

```

Diretório – lib/home/home_page.dart

```

import 'package:flutter/material.dart';
import '../services/esp32_wifi_service.dart';
import 'control_page.dart';

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  bool _loading = false;

  Future<void> _connect() async {
    setState(() => _loading = true);

    try {
      await Esp32WifiService.instance.connect();
    }
  }
}

```

```

    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text("Conectado ao ARMIAC")),
    );
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text("Erro: $e")),
    );
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}

```

```

Future<void> _send(String cmd) async {
  try {
    await Esp32WifiService.instance.action("close_hand");

```

```

    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text("Comando $cmd enviado ")),
    );
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text("Falha: $e")),
    );
  }
}

```

```

@override
Widget build(BuildContext context) {
  return ValueListenableBuilder<bool>(
    valueListenable: Esp32WifiService.instance.isConnected,
    builder: (context, connected, _) {
      return Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              connected ? "ARMIAC Conectado" : "ARMIAC Desconectado ",
              style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 20),

```

```

    SizedBox(
      width: double.infinity,
      child: ElevatedButton(
        onPressed: _loading ? null : _connect,
        child: Text(_loading ? "Conectando..." : "Conectar ao ARMIAC"),
      ),
    ),

    const SizedBox(height: 30),

    SizedBox(
      width: double.infinity,
      child: ElevatedButton(
        onPressed: () {
          Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const ControlPage()),
          );
        },
        child: const Text("Controle"),
      ),
    ),

    const SizedBox(height: 12),

    SizedBox(
      width: double.infinity,
      child: ElevatedButton(
        onPressed: connected ? () => _send("2") : null,
        child: const Text("Botão 2"),
      ),
    ),

    const SizedBox(height: 12),

    SizedBox(
      width: double.infinity,
      child: ElevatedButton(
        onPressed: connected ? () => _send("3") : null,
        child: const Text("Botão 3"),
      ),
    ),

    const SizedBox(height: 25),

    if (!connected)

```

```

        const Text(
          "Conecte o celular no Wi-Fi do ARMIAC (ARMAC_ESP32)",
          textAlign: TextAlign.center,
          style: TextStyle(color: Colors.grey),
        ),
      ],
    ),
  );
},
);
}
}

```

Diretório – lib/profile/profile_page.dart

```

import 'package:flutter/material.dart';

class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text("Página Perfil", style: TextStyle(fontSize: 24)),
    );
  }
}

```

Diretório – lib/services/esp32_wifi_service.dart

```

import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;

class Esp32WifiService {
  Esp32WifiService._();
  static final Esp32WifiService instance = Esp32WifiService._();

  static const String baseUrl = "http://192.168.4.1";
  final ValueNotifier<bool> isConnected = ValueNotifier<bool>(false);

  Future<void> connect() async {
    final ok = await ping();
    isConnected.value = ok;
    if (!ok) throw Exception("Conecte no Wi-Fi ARMIAC_ESP32 e tente novamente.");
  }

  Future<bool> ping() async {
    try {

```

```

    final r = await http
      .get(Uri.parse("$baseUrl/ping"))
      .timeout(const Duration(seconds: 2));
    return r.statusCode == 200;
  } catch (_) {
    return false;
  }
}

Future<void> action(String a) async {
  final r = await http
    .get(Uri.parse("$baseUrl/action?a=$a"))
    .timeout(const Duration(seconds: 3));
  if (r.statusCode != 200) {
    throw Exception("Falha ao enviar ação ($a): HTTP ${r.statusCode}");
  }
}
}

```

Diretório – lib/settings/settings_page.dart

```

import 'package:flutter/material.dart';
import 'package:package_info_plus/package_info_plus.dart';
import 'terms_page.dart';

class SettingsPage extends StatefulWidget {
  const SettingsPage({super.key});

  @override
  State<SettingsPage> createState() => _SettingsPageState();
}

class _SettingsPageState extends State<SettingsPage> {
  String appVersion = "";

  @override
  void initState() {
    super.initState();
    _loadVersion();
  }

  Future<void> _loadVersion() async {
    final info = await PackageInfo.fromPlatform();
    setState(() => appVersion = "${info.version} (${info.buildNumber})");
  }

  @override
  Widget build(BuildContext context) {

```

```

return Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      const Text("Configurações", style: TextStyle(fontSize: 24)),
      const SizedBox(height: 20),
      Text("Versão do app: $appVersion",
        style: const TextStyle(fontSize: 16, color: Colors.grey)),
      const SizedBox(height: 30),
      GestureDetector(
        onTap: () => Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => const TermsPage()),
        ),
        child: const Text(
          "Termos de serviço ",
          style: TextStyle(
            color: Colors.blue,
            decoration: TextDecoration.underline,
            fontSize: 18,
          ),
        ),
      ),
    ],
  ),
);
}

```

Diretório – lib/settings/terms_page.dart

```

import 'package:flutter/material.dart';

class TermsPage extends StatefulWidget {
  const TermsPage({super.key});

  @override
  State<TermsPage> createState() => _TermsPageState();
}

class _TermsPageState extends State<TermsPage>
  with SingleTickerProviderStateMixin {
  late final AnimationController _controller;
  late final Animation<double> _fade;
  late final Animation<Offset> _slide;

  @override
  void initState() {

```

```

super.initState();

_controller = AnimationController(
  vsync: this,
  duration: const Duration(milliseconds: 450),
);

_fade = CurvedAnimation(parent: _controller, curve: Curves.easeOut);
_slide = Tween<Offset>(
  begin: const Offset(0, 0.06),
  end: Offset.zero,
).animate(CurvedAnimation(
  parent: _controller,
  curve: Curves.easeOutCubic,
));

_controller.forward();
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

Widget _sectionTitle(String text) {
  return Padding(
    padding: const EdgeInsets.only(top: 18, bottom: 8),
    child: Text(
      text,
      style: const TextStyle(
        fontSize: 16,
        fontWeight: FontWeight.w800,
      ),
    ),
  );
}

Widget _paragraph(String text) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 10),
    child: Text(
      text,
      style: TextStyle(
        fontSize: 14.5,
        height: 1.45,
        color: Colors.grey.shade800,

```



```

    ),
  ),
);
}

```

```

Widget _bullets(List<String> items) {
  return Column(
    children: items.map((t) {
      return Padding(
        padding: const EdgeInsets.only(bottom: 8),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text("• ", style: TextStyle(fontSize: 16)),
            Expanded(
              child: Text(
                t,
                style: TextStyle(
                  fontSize: 14.5,
                  height: 1.45,
                  color: Colors.grey.shade800,
                ),
              ),
            ),
          ],
        ),
      ).toList(),
    );
}

```

```

@override
Widget build(BuildContext context) {
  final cs = Theme.of(context).colorScheme;

  return Scaffold(
    appBar: AppBar(
      title: const Text("Termos de Serviço"),
      backgroundColor: cs.inversePrimary,
    ),
    body: FadeTransition(
      opacity: _fade,
      child: SlideTransition(
        position: _slide,
        child: ListView(
          padding: const EdgeInsets.fromLTRB(16, 16, 16, 24),
          children: [

```

```

Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    borderRadius: BorderRadius.circular(18),
    gradient: LinearGradient(
      colors: [
        cs.primary.withOpacity(0.12),
        cs.secondary.withOpacity(0.10),
      ],
      begin: Alignment.topLeft,
      end: Alignment.bottomRight,
    ),
    border: Border.all(color: cs.primary.withOpacity(0.18)),
  ),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Text(
        "TERMOS DE SERVIÇO – ARMIAC (PROJETO INDIVIDUAL)",
        style:
          TextStyle(fontSize: 18, fontWeight: FontWeight.w900),
      ),
      const SizedBox(height: 8),
      Row(
        children: [
          Icon(Icons.event, size: 18, color: cs.primary),
          const SizedBox(width: 8),
          const Text(
            "Última atualização: Fevereiro de 2026",
            style: TextStyle(
              fontSize: 13.5, fontWeight: FontWeight.w600),
          ),
        ],
      ),
    ],
  ),
  const SizedBox(height: 18),

  _paragraph(
    "Bem-vindo(a) à ARMIAC. Estes Termos de Serviço ("Termos") regulam o  

    acesso e utilização "
    "da aplicação móvel ARMIAC ("Aplicação" ou "Serviço").",
  ),
  _paragraph(
    "Ao instalar, aceder ou utilizar a Aplicação, o utilizador declara que leu,  

    compreendeu e aceita "

```


"Caso o utilizador seja menor de idade, a utilização deve ser autorizada e supervisionada por um "

"responsável legal.",

),

_sectionTitle("5. CONTA E SEGURANÇA (SE APLICÁVEL)"),

_paragraph(
 "Se a aplicação permitir criação de conta, o utilizador é responsável por manter a palavra-passe "
 "segura e confidencial, não partilhar credenciais de acesso e comunicar qualquer suspeita de acesso "
 "indevido.",
),

_paragraph(
 "O Responsável não se responsabiliza por danos causados por acesso indevido resultante de negligência "
 "do utilizador.",
),

_sectionTitle("6. CONDUTA DO UTILIZADOR"),

_paragraph("O utilizador compromete-se a não utilizar a ARMIAC para:"),

_bullets([
 "atividades ilegais ou fraudulentas;",
 "violação de direitos de terceiros;",
 "envio ou publicação de conteúdos ofensivos, difamatórios, discriminatórios ou violentos;",
 "disseminação de malware, vírus ou tentativas de ataque informático;",
 "tentativa de acesso não autorizado a sistemas ou dados.",
]),

_paragraph("O uso indevido pode resultar na suspensão ou bloqueio do acesso ao Serviço."),

_sectionTitle("7. PROPRIEDADE INTELECTUAL"),

_paragraph(
 "O nome ARMIAC, logotipo, design, código-fonte, estrutura, funcionalidades e identidade visual são "
 "propriedade do Responsável pelo Serviço, estando protegidos por legislação de direitos de autor e "
 "propriedade intelectual.",
),

_paragraph(
 "É proibida a reprodução, modificação, distribuição ou exploração comercial da aplicação ou do seu "
 "conteúdo sem autorização prévia por escrito.",
),

_sectionTitle("8. CONTEÚDO INSERIDO PELO UTILIZADOR"),

```

    _paragraph(
        "O utilizador é o único responsável pelo conteúdo que inserir na aplicação,
garantindo que possui "
        "direitos sobre os dados ou autorização para os utilizar, não viola direitos de
terceiros e não publica "
        "conteúdos ilegais.",
    ),
    _paragraph(
        "O Responsável reserva-se o direito de remover conteúdos que violem estes
Termos ou a legislação aplicável.",
    ),

    _sectionTitle("9. PRIVACIDADE E PROTEÇÃO DE DADOS (RGPD)"),
    _paragraph(
        "A ARMIAC respeita a privacidade do utilizador e compromete-se a tratar os
dados pessoais com segurança.",
    ),
    _paragraph(
        "O tratamento de dados pessoais é realizado conforme o Regulamento (UE)
2016/679 (RGPD) e a Lei n.º 58/2019, "
        "de 8 de agosto (Portugal).",
    ),
    _paragraph(
        "Os dados recolhidos podem incluir informações necessárias para
funcionamento da aplicação, melhoria do serviço "
        "e suporte ao utilizador.",
    ),

    _sectionTitle("10. DISPONIBILIDADE E MANUTENÇÃO"),
    _paragraph(
        "O Responsável procura manter a aplicação disponível, mas não garante
funcionamento contínuo ou livre de falhas.",
    ),
    _paragraph("A aplicação pode ficar temporariamente indisponível devido a:"),
    _bullets([
        "manutenção técnica;",
        "atualizações;",
        "falhas de internet ou servidores;",
        "motivos de segurança;",
        "eventos de força maior."
    ]),

    _sectionTitle("11. LIMITAÇÃO DE RESPONSABILIDADE"),
    _paragraph("Na extensão permitida pela legislação aplicável, o Responsável não
será responsável por:"),
    _bullets([
        "perdas indiretas ou lucros cessantes;"
    ]),

```

"danos decorrentes de falhas técnicas, indisponibilidade ou interrupções;",
 "problemas causados por dispositivos do utilizador;",
 "uso indevido do Serviço por terceiros;",
 "erros resultantes de informações inseridas pelo utilizador.",
),
 _paragraph("O utilizador reconhece que utiliza a aplicação por sua conta e
 risco."),

 _sectionTitle("12. SERVIÇOS PAGOS (SE APLICÁVEL)",
 _paragraph(
 "Caso a ARMIAC venha a disponibilizar funcionalidades pagas, os preços e
 condições serão apresentados "
 "antes da contratação.",
),
 _paragraph(
 "O utilizador concorda que os valores podem ser atualizados, poderão existir
 planos recorrentes e reembolsos "
 "seguirão as regras da plataforma (Google Play / App Store) e legislação
 aplicável.",
),

 _sectionTitle("13. CANCELAMENTO E ELIMINAÇÃO DE CONTA"),
 _paragraph("O utilizador pode deixar de utilizar a aplicação a qualquer
 momento."),
 _paragraph(
 "Quando aplicável, o utilizador poderá solicitar a eliminação de conta e dados
 através do contacto oficial "
 "disponibilizado.",
),
 _paragraph(
 "O Responsável pode suspender ou eliminar contas em caso de violação destes
 Termos ou exigência legal.",
),

 _sectionTitle("14. ALTERAÇÕES DOS TERMOS"),
 _paragraph(
 "O Responsável pode atualizar estes Termos periodicamente. Caso existam
 alterações relevantes, o utilizador "
 "poderá ser informado através da aplicação.",
),
 _paragraph("O uso contínuo após atualização significa aceitação dos novos
 Termos."),

 _sectionTitle("15. LEI APLICÁVEL E FORO"),
 _paragraph("Estes Termos são regidos pela legislação portuguesa e legislação
 aplicável da União Europeia."),
 _paragraph(

"Para resolução de litígios, será competente o foro da comarca do domicílio do Responsável, salvo norma legal "

"imperativa em contrário.",

),

_sectionTitle("16. CONTACTO"),

_paragraph("Para dúvidas, suporte ou solicitações relacionadas com estes Termos, o utilizador poderá contactar:"),

_bullets([

"Email: kevinleonardomail@gmail.com",

]),

const SizedBox(height: 18),

Container(

padding: const EdgeInsets.all(14),

decoration: BoxDecoration(

borderRadius: BorderRadius.circular(16),

color: Colors.grey.shade100,

border: Border.all(color: Colors.grey.shade300),

),

child: const Column(

crossAxisAlignment: CrossAxisAlignment.start,

children: [

Text(

"DECLARAÇÃO FINAL",

style: TextStyle(fontSize: 15, fontWeight: FontWeight.w900),

),

SizedBox(height: 8),

Text(

"Ao utilizar a aplicação ARMIAC, o utilizador declara que leu e concorda com os presentes Termos de Serviço.",

style: TextStyle(fontSize: 14.5, height: 1.4),

),

],

),

),

],

),

),

),

);

}

}

4.4.1 – Gestão de dependências e recursos (pubspec.yaml)

O ficheiro **pubspec.yaml** é um dos ficheiros mais importantes de um projeto Flutter, pois é nele que se definem:

- as dependências externas (packages),
- a versão do SDK,
- os recursos do projeto (assets), como imagens,
- e informações básicas da aplicação (nome e versão).

No ARMIAC, o **pubspec.yaml** foi utilizado para integrar bibliotecas essenciais:

- **http** – permite realizar pedidos HTTP ao ESP32 (utilizado nos endpoints **/ping** e **/action**);
- **package_info_plus** – utilizado para obter e apresentar a versão da aplicação na área de configurações;
- **assets (imagens)** – utilizado para carregar o logótipo do projeto dentro da aplicação.

A definição correta das dependências e dos assets no **pubspec.yaml** foi fundamental para garantir que a aplicação funcionasse corretamente e que a imagem do logótipo fosse carregada no topo do aplicativo.

Código pubspec.yaml:

Diretório - /pubspec.yaml

```
name: proj_armiac
description: "Desenvolvido por Quévin Tavares"

publish_to: 'none'

version: 1.0.0+1

environment:
  sdk: ">=3.10.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  package_info_plus: ^8.0.2
```



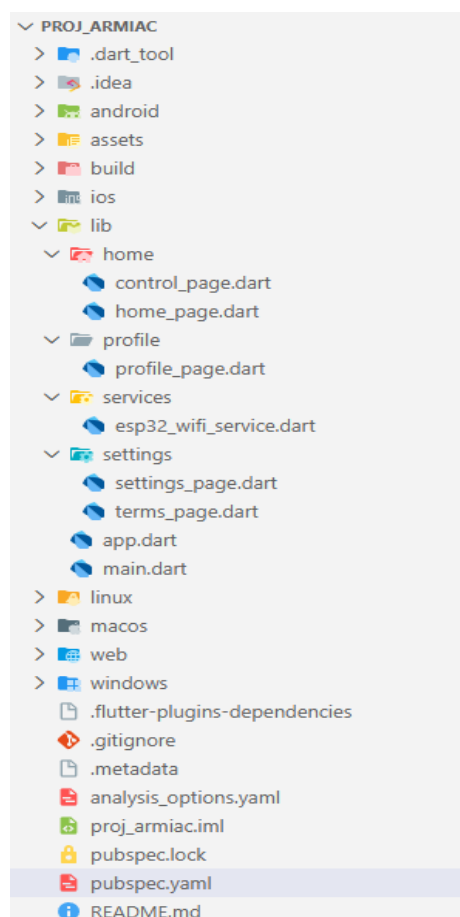
```
http: ^1.2.2
dev_dependencies:
  flutter_test:
    sdk: flutter
flutter_lints: ^6.0.0
```

```
flutter:
  uses-material-design: true
assets:
  - assets/images/Armiac_LOGO.png
```

4.4.2 – Organização do projeto Flutter (estrutura de pastas)

Durante o desenvolvimento da aplicação ARMIAC foi adotada uma organização por pastas, com o objetivo de manter o código mais limpo, modular e fácil de manter. A separação dos ficheiros permite distinguir claramente:

- páginas da interface (UI),
- serviços de comunicação com o ESP32,
- e outras funcionalidades como definições/termos.

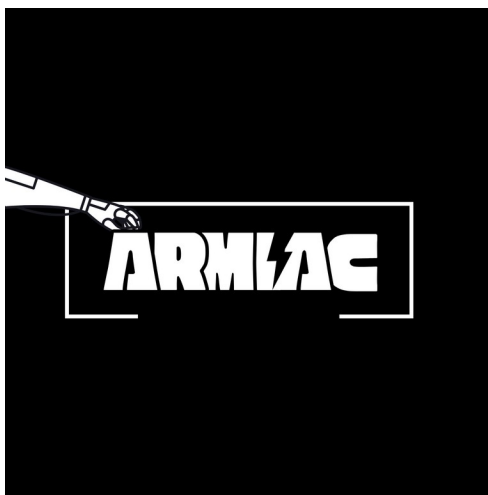


4.5 – Identidade Visual do Projeto (Logótipo ARMIAC)

Para dar uma identidade visual ao projeto, foi desenvolvido um logótipo com o nome **ARMIAAC**. O logótipo foi criado utilizando o software **Canva**, permitindo criar uma imagem representativa do braço robótico e do nome do projeto.

O logótipo foi posteriormente integrado na aplicação Flutter, sendo apresentado no topo da interface principal.

Figura 38: Logo ARMIAC



4.6 – Dificuldades encontradas durante o desenvolvimento

Durante o desenvolvimento do projeto **ARMIAAC** surgiram várias dificuldades técnicas que exigiram pesquisa, testes e adaptação de soluções, tanto na parte física como na parte de software.

Uma das principais dificuldades foi relacionada com a **impressão 3D** das peças. Em algumas tentativas ocorreram falhas nos modelos e erros durante a impressão, como deformações e problemas de aderência à base, o que levou à repetição de várias impressões e ajustes nos ficheiros STL e nas configurações de slicing. Este processo aumentou o tempo de desenvolvimento, mas permitiu compreender melhor a importância da preparação correta dos modelos e das definições da impressora **Elegoo Neptune 4**.

Outra dificuldade relevante foi a **instalação e configuração do Flutter** no ambiente de desenvolvimento. Durante este processo, ocorreram erros em que o Flutter não era reconhecido corretamente pelo sistema, devido a problemas no PATH (variável de ambiente). O erro mais frequente era o sistema não conseguir localizar o comando “flutter”, impedindo a execução de comandos no terminal. Para resolver este problema foi

necessário configurar manualmente o PATH, garantindo que a pasta **bin** do Flutter estava corretamente adicionada às variáveis do sistema, permitindo assim que o Flutter fosse executado corretamente.

Além disso, apesar de não ter sido exatamente uma dificuldade, foi necessário **aprender a linguagem Dart** desde o início, uma vez que não tinha experiência prévia com esta linguagem. Foi necessário estudar a sintaxe, a estrutura de projetos Flutter, e conceitos como widgets, navegação entre páginas e integração de packages.

Esta aprendizagem foi essencial para conseguir desenvolver uma aplicação funcional e estável para controlar o protótipo.

Apesar destas dificuldades, todas foram ultrapassadas através de pesquisa, testes e persistência, contribuindo para uma evolução significativa das minhas competências técnicas ao longo do projeto.

5 – Conclusão

O desenvolvimento do projeto **ARMAC** permitiu aplicar conhecimentos adquiridos ao longo do curso, combinando áreas como **programação, eletrônica, redes e impressão 3D**. Foi possível criar um **protótipo funcional de uma mão robótica**, capaz de realizar movimentos básicos e gestos, sendo controlada remotamente por uma aplicação móvel através de Wi-Fi.

Durante o desenvolvimento surgiram várias dificuldades técnicas. Uma das principais dificuldades esteve relacionada com a **impressão 3D**, devido a falhas nos modelos e erros durante algumas impressões, o que obrigou a repetir peças e ajustar configurações para garantir um resultado final correto. Também foram encontrados obstáculos na **instalação e configuração do Flutter**, principalmente devido a problemas com o PATH do sistema, que impedia o reconhecimento correto do comando “flutter” no terminal. Além disso, foi necessário **aprender a linguagem Dart desde o início**, uma vez que não havia experiência prévia com esta linguagem, o que exigiu estudo e prática para conseguir desenvolver a aplicação.

Apesar do protótipo estar funcional, existem **melhorias futuras** possíveis, como:

- Substituição da fonte por uma bateria 7.4V recarregável com conversor step-down;
- Criação de controlo mais detalhado na aplicação através de sliders;
- Completamento restante do braço;
- Implementação de sensores de força e posição para melhorar precisão e segurança.

Concluindo, o projeto **ARMAC** foi desenvolvido com sucesso e representa um trabalho completo que integra várias áreas técnicas, demonstrando **competências essenciais adquiridas durante o curso**.

6 – Bibliografia/ Webgrafia

- Arduino Documentation – <https://www.arduino.cc>
- Espressif ESP32 Documentation – <https://www.espressif.com>
- Adafruit PCA9685 Documentation – <https://learn.adafruit.com>
- Flutter Documentation – <https://docs.flutter.dev>
- Dart Documentation – <https://dart.dev>
- HTTP Package (Flutter) – <https://pub.dev/packages/http>
- Canva – <https://www.canva.com>
- Servo MG996R - [MG996R.pdf](#)

7 – Anexos

Anexos ao relatório...

